

Tecnicatura Superior en Desarrollo de Software

Anexo 05c — ¿Qué colección usar? Criterios de selección en C#

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Por qué importa la elección?

Conocer las colecciones disponibles es solo el primer paso. El verdadero desafío profesional es elegir la estructura correcta para cada problema. Una elección incorrecta puede producir código que funciona pero que degrada el rendimiento, es difícil de mantener, o expone datos que deberían protegerse.

Este anexo presenta un proceso de decisión sistemático basado en cuatro preguntas clave, seguido de casos de uso reales donde se aplica el razonamiento completo.

Las cuatro preguntas para elegir una colección



Casos de uso reales

Caso 1 Sistema de gestión de alumnos Colección elegida: Dictionary<string, Alumno>

Problema: Una aplicación necesita buscar el legajo de un alumno por número de DNI varias veces por segundo durante el proceso de inscripción.

Razonamiento: La operación dominante es *buscar por identificador conocido*. Con una lista de 5.000 alumnos, cada búsqueda lineal revisa en promedio 2.500 registros. Un diccionario resuelve cada búsqueda en tiempo constante.

- Clave: DNI (string)
- Valor: objeto Alumno
- Búsqueda: $O(1)$
- Interfaz: IDictionary<string, Alumno>

```
1 private IDictionary<string, Alumno> _alumnos = new Dictionary<string, Alumno>();
```

```
2
3 public Alumno BuscarPorDni(string dni)
4 {
5     if (!_alumnos.TryGetValue(dni, out var alumno))
6         throw new KeyNotFoundException($"DNI {dni} no registrado.");
7     return alumno;
8 }
```

Caso 2 Historial de páginas visitadas (sin repetidos) Colección elegida:
HashSet<string>

Problema: Un navegador web debe registrar las URLs visitadas para marcarlas como "ya visitadas" al renderizar nuevos enlaces. No importa cuántas veces se visitó, solo si fue visitada o no.

Razonamiento: Solo se necesita verificar pertenencia y garantizar unicidad. No hay valor asociado a cada URL, y el orden no importa.

- Elemento: URL (string)
- Sin duplicados automático
- Contains: O(1)
- Interfaz: ISet<string>

```
1 private readonly HashSet<string> _visitadas = new();
2
3 public void Registrar(string url) => _visitadas.Add(url);
4 public bool FueVisitada(string url) => _visitadas.Contains(url);
```

Caso 3 Lista de productos de un carrito de compras Colección elegida:
List<LineaCarrito>

Problema: Un carrito de compras necesita mostrar productos en el orden en que se agregaron, permitir agregar, quitar y acceder por posición para mostrar numeración al usuario.

Razonamiento: Se necesita orden de inserción, duplicados permitidos (el mismo producto puede aparecer varias veces), acceso por índice para la vista y mutación dinámica.

- Orden garantizado
- Acceso por índice
- Permite duplicados
- Interfaz: IList<LineaCarrito>

Caso 4 Método que procesa cualquier secuencia de nombres Parámetro elegido: `IEnumerable<string>`

Problema: Un método de utilidad debe imprimir un reporte a partir de una lista de nombres. El origen puede ser un arreglo, una lista, un resultado de base de datos o un archivo CSV.

Razonamiento: El método solo itera, no modifica, no accede por índice. Declarar el parámetro como `IEnumerable<string>` lo hace compatible con cualquier origen de datos presente y futuro.

- Solo iteración
- Sin modificación
- Máxima compatibilidad
- Principio: mínima dependencia

```

1 // Acepta T[], List<T>, resultados de LINQ, etc.
2 public void GenerarReporte(IEnumerable<string> nombres)
3 {
4     Console.WriteLine("=== Reporte ===");
5     foreach (var nombre in nombres)
6         Console.WriteLine($"- {nombre}");
7 }

```

Tabla de referencia rápida

Colección	Búsqueda	Inserción	Orden	Duplicados	Cuándo elegirla
<code>T[]</code>	$O(1)$ por índice	No (tamaño fijo)	Garantizado	Si	Tamaño conocido y fijo, máximo rendimiento
<code>List<T></code>	$O(1)$ por índice / $O(n)$ por valor	$O(1)$ al final*	Garantizado	Si	Colección dinámica con acceso posicional

Colección	Búsqueda	Inserción	Orden	Duplicados	Cuándo elegirla
Dictionary<K,V>	$O(1)^*$	$O(1)^*$	No garantizado	No — Claves únicas	Búsqueda por clave conocida
HashSet<T>	$O(1)^*$	$O(1)^*$	No garantizado	No	Unicidad, pertenencia, operaciones de conjunto
SortedDictionary<K,V>	$O(\log n)$	$O(\log n)$	Por clave	No — Claves únicas	Búsqueda por clave + iteración ordenada
ICollection<T>	$O(1)$ por índice	Sin mutación	Garantizado	Si	Exponer colección sin permitir modificación externa

* $O(1)$ promedio. Puede degradar a $O(n)$ en colisiones extremas.

Errores de diseño frecuentes en la selección

ERROR DE DISEÑO

Usar `List<T>` para todo, incluso cuando no se necesita orden ni acceso por índice.

```

1 // Lista de permisos activos – solo se necesita "¿tiene el permiso?"
2 var permisos = new List<string> { "leer", "escribir", "admin" };
3 if (permisos.Contains("admin")) { } //  $O(n)$ 
```

La lista funciona correctamente pero es la herramienta equivocada. A medida que crece la cantidad de permisos, el costo de verificación crece linealmente.

DISEÑO CORRECTO

Usar `HashSet<T>` cuando la operación es verificar pertenencia.

```
1 var permisos = new HashSet<string> { "leer", "escribir", "admin" };
2 if (permisos.Contains("admin")) { } // O(1)
```

Independientemente de la cantidad de permisos, la verificación tiene costo constante. El código expresa además la intención: un conjunto de permisos únicos, no una secuencia ordenada.



Principio de mínima interfaz: En parámetros y retornos de métodos, siempre declarar el tipo más abstracto que satisfaga los requisitos. Si el método solo necesita iterar, usar `IEnumerable<T>`. Si necesita Count y Add, usar `ICollection<T>`. Si necesita acceso por índice, usar `IList<T>`. Nunca exponer `List<T>` concreto en firmas públicas si puede evitarse.

Ejercicio integrador

Dado el siguiente escenario, identificar qué colección corresponde a cada necesidad y justificar:

Escenario: Una plataforma de e-learning necesita: (a) almacenar los cursos disponibles en orden de publicación para mostrarlos paginados, (b) registrar qué alumnos ya completaron cada curso para no mostrarles el botón "inscribirse", (c) buscar los datos de un alumno por su email al momento del login, (d) recibir en un método utilitario cualquier fuente de datos de cursos para generar un PDF.

Necesidad	Colección recomendada	Justificación
(a) Cursos paginados	<code>List<Curso></code>	Orden garantizado + acceso por índice para paginación
(b) Alumnos que completaron	<code>HashSet<string></code>	Solo pertenencia, sin valor adicional, sin duplicados
(c) Buscar alumno por email	<code>Dictionary<string, Alumno></code>	Búsqueda por clave (email) en O(1)



Necesidad	Colección recomendada	Justificación
(d) Parámetro de generación PDF	<code>IEnumerable<Curso></code>	Solo iteración, máxima compatibilidad con cualquier origen